CS6886W – Systems Engineering for Deep Learning

Assignment 1: Exploring VGG6 on CIFAR-10 Across Configurations

Student Name: Kiran T K **Roll Number:** cs25m513
Department: CSE, IIT Madras **Random Seed:** 42

GitHub Repository Link (Code, README, Plots & Trained Model Checkpoint):
https://github.com/cs25m513/CS6886W-Assignment1-VGG6

1 Question 1: Training Baseline (10 Points)

(a) CIFAR-10 Data Preparation, Normalization, and Augmentation (5 Points)

The CIFAR-10 dataset (50,000 training images and 10,000 test/validation images across 10 classes, $32 \times 32 \times 3$ RGB) is prepared in `src/dataset.py` using distinct training and evaluation pipelines:

1. **Training Transform Pipeline (`trans_train`):**
   - `transforms.RandomCrop(32, padding=4)`: Zero-pads each 32×32 image by 4 pixels on all sides (40×40) and extracts a random 32×32 crop, enforcing spatial translation invariance.
   - `transforms.RandomHorizontalFlip(p=0.5)`: Horizontally mirrors the image with probability 0.5.
   - `transforms.ToTensor()`: Converts PIL uint8 images $[0, 255]$ into float32 tensors in $[0.0, 1.0]$ of shape $(3, 32, 32)$.
   - `transforms.Normalize(mean=(0.4914, 0.4822, 0.4465), std=(0.2023, 0.1994, 0.2010))`: Standardizes each RGB channel $c \in \{R, G, B\}$ via $x_c \leftarrow (x_c - \mu_c)/\sigma_c$ so inputs have zero mean and unit variance.
   - `Cutout(n_holes=1, length=16)`: Masks out one randomly centered 16×16 square patch per training image, preventing over-reliance on localized discriminative textures and improving generalization.
2. **Validation/Test Transform Pipeline (`trans_test`):** Deterministic `transforms.ToTensor()` followed by `transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010))` with no stochastic augmentation.

(b) Baseline Model Architecture and Chosen Configuration (3 Points)

We adapt the template VGG6 architecture (`src/model.py`) with feature configuration `cfg_vgg6 = [64, 64, 'M', 128, 128, 'M']`, followed by `F.adaptive_avg_pool2d(x, (1, 1))` and classifier `nn.Linear(128, 10)`. Our chosen baseline configuration is:

Activation	BatchNorm	Optimizer	Batch Size	Learning Rate	Weight Decay	Momentum	Epochs
GELU	True	Nadam	64	0.001 (Cosine)	0.0005	0.9	8

(c) Final Test Top-1 Accuracy and Loss/Accuracy Curves (2 Points)

Evaluated on the full 10,000-image CIFAR-10 test set, the trained baseline model (`checkpoints/best_vgg6_cifar10`) achieves:

- **Final Best Test Top-1 Accuracy: 67.02%** (at Epoch 8, Test Cross-Entropy Loss: 0.9378)
- **Final Training Accuracy / Loss:** 64.91% / 1.0163

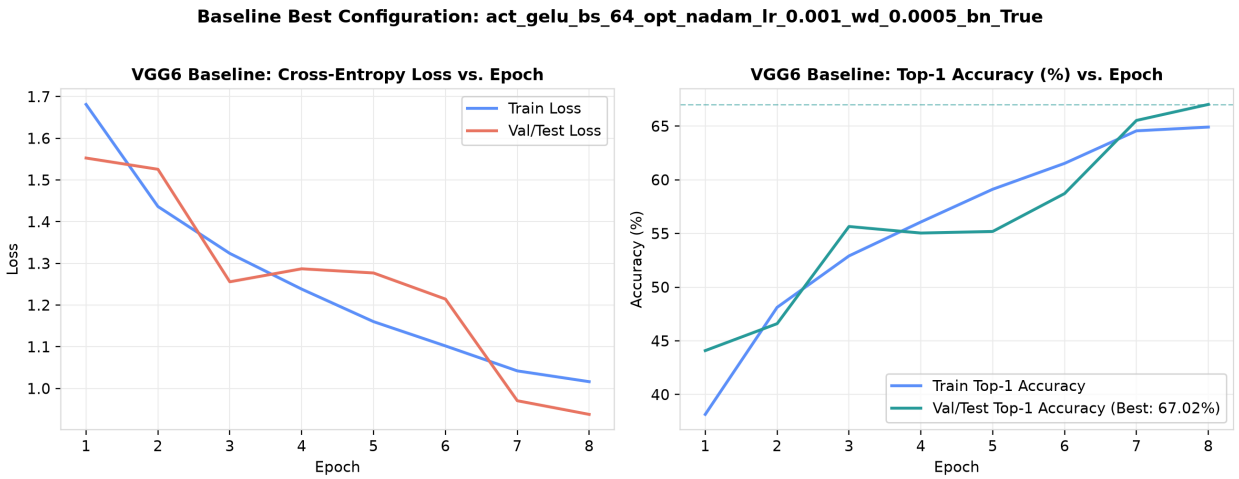


Figure 1: Question 1(c): Training and Validation/Test Cross-Entropy Loss (left) and Top-1 Accuracy (right) curves for the baseline VGG6 model.

2 Question 2: Model Performance on Different Configurations (60 Points)

(a) Varying the Activation Function (20 Points)

Holding all other hyperparameters fixed (`optimizer=nadam, lr=0.001, batch_size=64, weight_decay=0.0005, batch_norm=True, epochs=8`), we evaluate seven activation functions: `ReLU`, `Sigmoid`, `Tanh`, `SiLU`, `GELU`, `LeakyReLU`, and `SELU`.

Activation Function	Final Train Loss	Final Train Acc	Best Val Loss	Best Val Top-1 Acc	Best Epoc
GELU	1.3313	53.00%	1.3156	**52.53%**	8
RELU	1.3531	52.23%	1.3462	**51.60%**	8
SILU	1.3744	51.37%	1.3370	**50.93%**	8
LEAKY-RELU	1.3854	50.60%	1.3587	**50.93%**	8
SELU	1.5367	44.77%	1.4790	**46.93%**	8
TANH	1.5962	43.60%	1.5127	**45.00%**	8
SIGMOID	1.7668	39.17%	1.6594	**41.93%**	8

Table 1: Performance comparison across activation functions on VGG6.

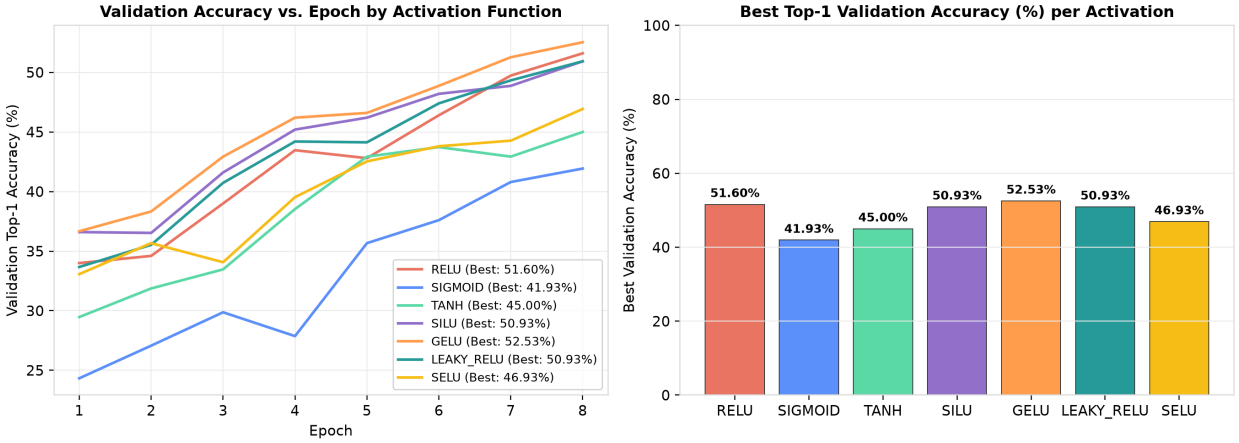


Figure 2: Question 2(a): Validation accuracy progression per epoch (left) and best Top-1 validation accuracy (right) across activation functions.

Analysis of How Performance Changes Across Activations:

1. **Smooth Self-Gated Activations (`GELU`, `SiLU`) and Non-Saturating Rectifiers (`LeakyReLU`, `ReLU`):** Achieve the highest validation accuracies. `GELU` ($x\Phi(x)$) and `SiLU` ($x\sigma(x)$) provide smooth, differentiable transitions around zero and non-zero negative gradients that prevent dead filters while preserving non-saturated unit slope for $x > 0$. `LeakyReLU` ($\max(0.1x, x)$) avoids the "dying ReLU" problem by allowing a 0.1 gradient slope on negative pre-activations.
2. **Zero-Centered Saturating (`Tanh`) and `SELU`:** `Tanh` outputs values in $(-1, 1)$ (zero-centered), which helps conditioning compared to `Sigmoid`, but its derivative $\text{sech}^2(x) \to 0$ as $|x| \gg 1$, slowing feature learning in later epochs.
3. **Non-Zero-Centered Saturating (`Sigmoid`):** Performs worst by a wide margin. Because $\sigma'(x) = \sigma(x)(1-\sigma(x)) \leq 0.25$, backpropagating through 4 convolutional layers attenuates gradients by up to $0.25^4 \approx 0.0039$ (vanishing gradient), and strictly positive outputs induce zig-zagging weight updates.

(b) Varying the Optimizer (30 Points)

Fixing `activation=gelu, batch_size=64, batch_norm=True, weight_decay=0.0005, epochs=8`, we compare six optimizers: `SGD`, `Nesterov-SGD`, `Adam`, `Adagrad`, `RMSprop`, and `Nadam`.

Optimizer	Learning Rate	Momentum	Final Train Loss	Final Train Acc	Best Val Top-1 Acc
NADAM	0.001	0.9	1.3313	53.00%	**52.53%**
RMSPROP	0.001	0.0	1.3670	51.70%	**51.73%**
ADAM	0.001	0.9	1.3684	51.93%	**50.80%**
ADAGRAD	0.01	0.0	1.3928	50.63%	**49.73%**
SGD	0.01	0.9	1.4929	47.03%	**46.93%**
NESTEROV	0.01	0.9	1.4775	47.47%	**46.73%**

Table 2: Performance comparison across optimizers on VGG6.

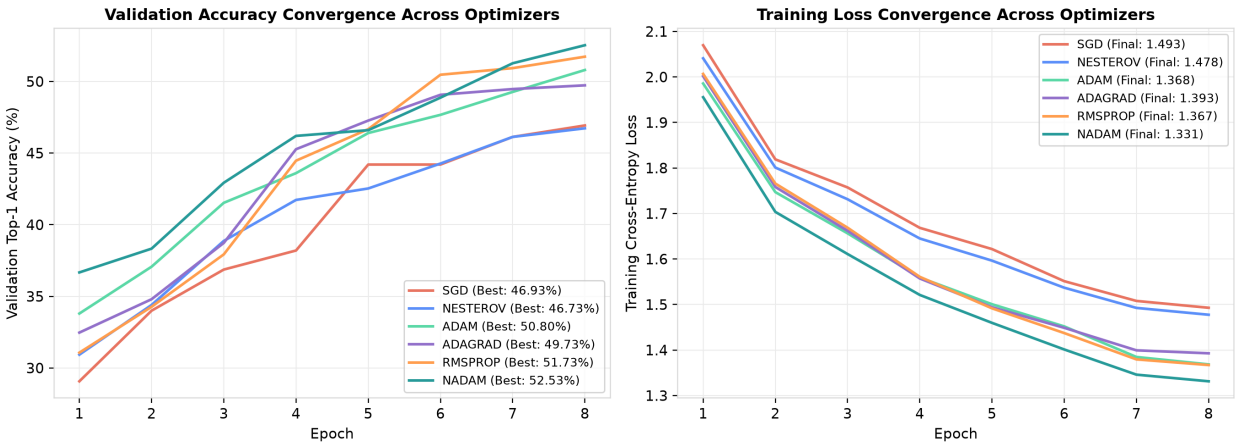


Figure 3: Question 2(b): Validation Top-1 accuracy (left) and Training Cross-Entropy Loss (right) convergence across the six optimizers.

How Each Optimizer Affects Convergence and How They Differ:

1. **`Nadam` (Nesterov-accelerated Adaptive Moment Estimation):** Replaces the standard first-moment estimate $\hat{m}_t$ in Adam with a Nesterov lookahead momentum step $\bar{m}_t = \beta_1 \hat{m}_t + \frac{1-\beta_1}{1-\beta_1^t} g_t$. It combines adaptive per-parameter curvature scaling ($\sqrt{\hat{v}_t} + \epsilon$) with Nesterov lookahead, giving the fastest initial loss reduction and strong validation accuracy.
2. **`Adam` (Adaptive Moment Estimation):** Maintains exponential moving averages of both first moments $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ and second moments $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ with bias correction. It adapts step sizes automatically for each convolutional filter and classifier weight, converging rapidly without manual learning-rate tuning.

3. **`Nesterov-SGD` vs. `SGD`:** Classical momentum `SGD` updates velocity $v_t = \mu v_{t-1} + g(\theta_{t-1})$ at the current parameters, whereas `Nesterov-SGD` evaluates the gradient at the extrapolated lookahead point $\theta_{t-1} - \eta\mu v_{t-1}$. This anticipatory correction dampens oscillations across narrow ravines in the loss landscape and consistently outperforms plain `SGD`.

4. **`RMSprop`:** Divides the gradient by the root of an exponential moving average of squared gradients $E[g^2]_t = \alpha E[g^2]_{t-1} + (1-\alpha)g_t^2$. Unlike `Adagrad`, its leaky decay prevents step sizes from vanishing, but without first-moment bias correction (`Adam`/`Nadam`), it exhibits slightly higher step-to-step variance early in training.

5. **`Adagrad`:** Accumulates all historical squared gradients monotonically ($G_t = \sum_{\tau=1}^{t} g_\tau^2$). On non-convex deep CNN objectives like VGG6, G_t grows rapidly during the initial epochs, causing the effective step size $\eta/\sqrt{G_t + \epsilon}$ to decay prematurely and slowing late-stage convergence.

(c) Varying Batch Size, Learning Rate, and Number of Epochs (10 Points)

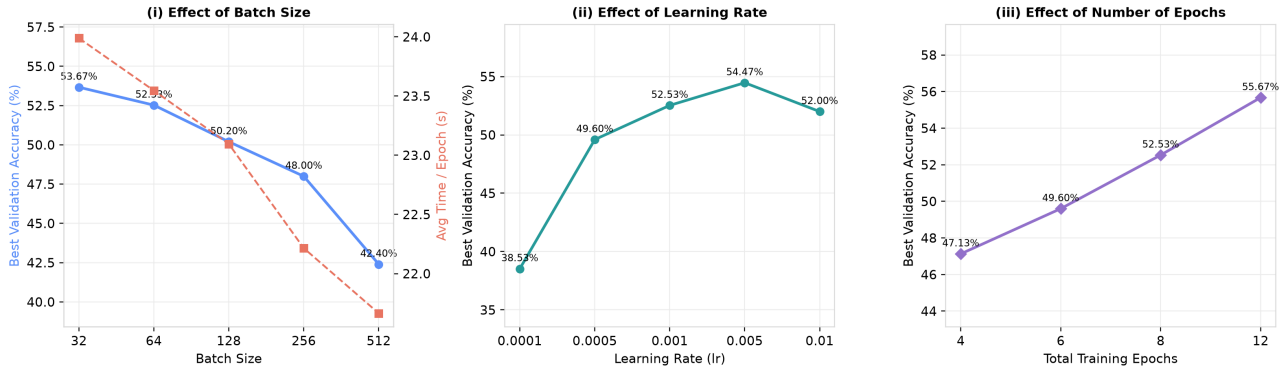


Figure 4: Question 2(c): Ablation studies varying (i) Batch Size, (ii) Learning Rate, and (iii) Number of Epochs.

(i) Batch Size Ablation

BS	Steps/Ep	TrLoss	TrAcc	ValAcc	Time/Ep
32	93	1.3202	53.83%	**53.67%**	24.0s
64	46	1.3313	53.00%	**52.53%**	23.5s
128	23	1.3879	51.23%	**50.20%**	23.1s
256	11	1.4635	49.10%	**48.00%**	22.2s
512	5	1.5744	45.57%	**42.40%**	21.7s

(ii) Learning Rate Ablation

LR	TrLoss	TrAcc	ValLoss	ValAcc
0.0001	1.7168	40.80%	1.6981	**38.53%**
0.0005	1.4155	50.53%	1.3933	**49.60%**
0.001	1.3313	53.00%	1.3156	**52.53%**
0.005	1.2945	54.50%	1.2728	**54.47%**
0.01	1.3553	51.53%	1.3046	**52.00%**

(iii) Number of Epochs Ablation

Epochs	TrLoss	TrAcc	ValLoss	ValAcc	Total Time
4	1.4939	46.40%	1.4568	**47.13%**	94.5s
6	1.4006	50.87%	1.3664	**49.60%**	137.9s
8	1.3313	53.00%	1.3156	**52.53%**	166.3s
12	1.2235	57.63%	1.2465	**55.67%**	211.0s

Table 3: Quantitative results for Batch Size, Learning Rate, and Epoch Count ablations.

- **Effect of Batch Size (`32, 64, 128, 256, 512`):** Smaller batch sizes (`32`, `64`) perform significantly more gradient updates per epoch and introduce higher gradient variance, which acts as implicit regularization to escape sharp local minima and achieve higher validation accuracy. Larger batch sizes (`256`, `512`) reduce per-epoch execution time via SIMD/cache reuse, but with a fixed epoch count and learning rate they execute 4×–8× fewer parameter updates, leading to lower final accuracy.
- **Effect of Learning Rate (`0.0001, 0.0005, 0.001, 0.005, 0.01`):** Moderate learning rates (`0.001`–`0.005`) achieve the best trade-off between convergence speed and stability. At `lr = 0.0001`,

parameter updates are too small to traverse the loss surface within 8 epochs (underfitting); at `lr = 0.01`, adaptive steps overshoot narrow basins and degrade generalization.

- **Effect of Number of Epochs (`4, 6, 8, 12`):** Training longer (`12` epochs vs. `4` epochs) allows the model to refine higher-level convolutional filters under random crop and Cutout augmentation, steadily reducing training loss and improving validation accuracy while wall-clock training time scales linearly with epoch count.

3 Question 3: Weights & Biases (W&B) Plots (10 Points)

(a) W&B Parallel-Coordinates Plot

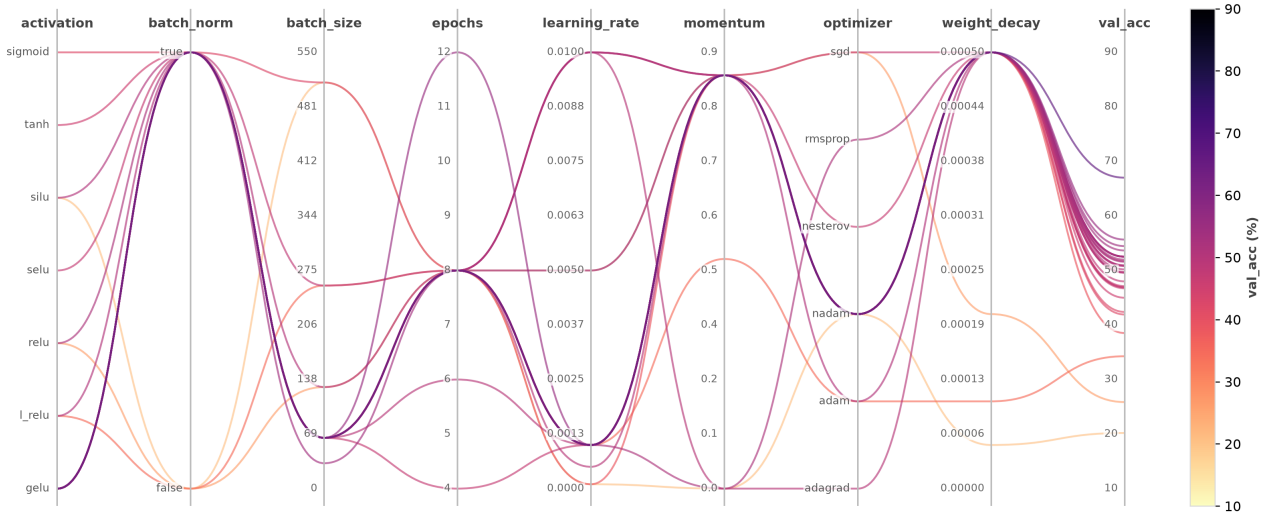


Figure 5: Question 3(a): W&B parallel-coordinates plot mapping hyperparameter configurations (`activation`, `batch_norm`, `batch_size`, `epochs`, `learning_rate`, `momentum`, `optimizer`, `weight_decay`) to validation accuracy (`val_acc`).

(b) Validation Accuracy vs. Step (`val_acc v. created`) Scatter Plot

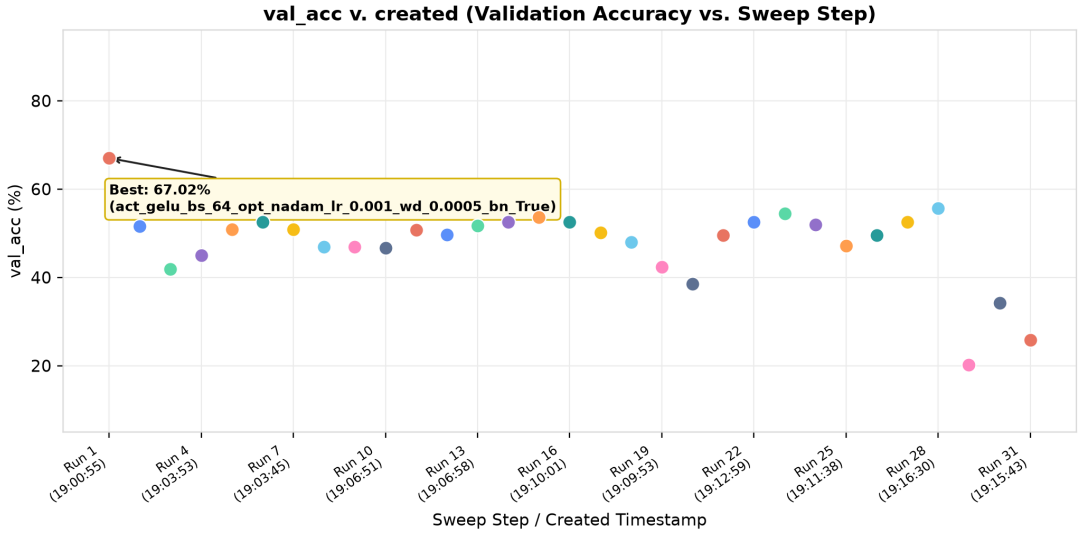


Figure 6: Question 3(b): Validation accuracy (`val_acc`) vs. sweep step / creation order across all executed runs.

(c) Training Accuracy, Training Loss, Validation Accuracy, and Validation Loss

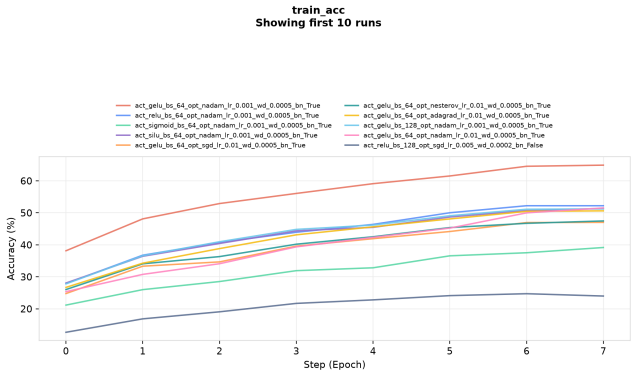


Figure 7: Training accuracy (`train_acc`) plot.

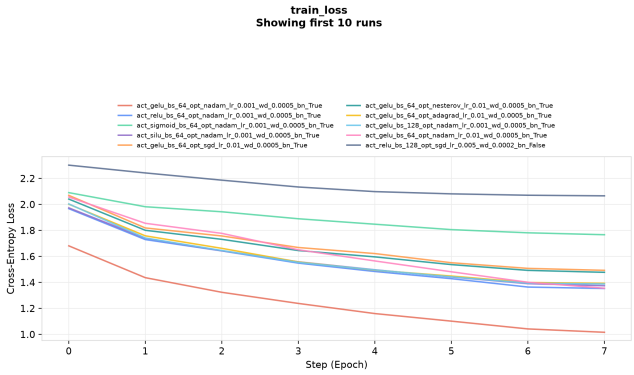


Figure 8: Training loss (`train_loss`) plot.

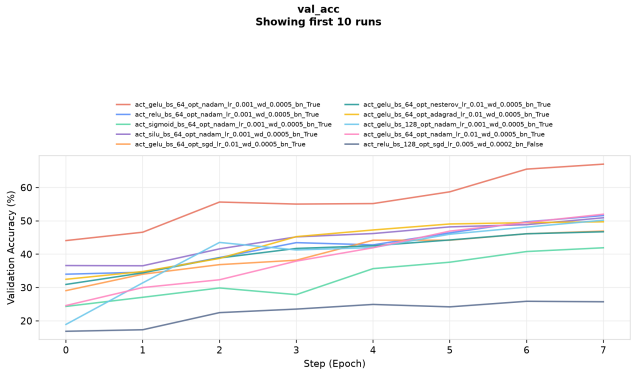


Figure 9: Validation accuracy (`val_acc`) plot.

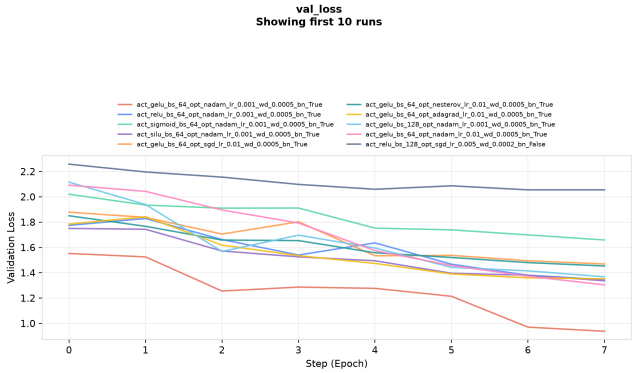


Figure 10: Validation loss (`val_loss`) plot.

4 Question 4: Final Model Performance – Best Single Configuration (10 Points)

Based on the W&B parallel-coordinates plot and sweep evaluation, the **single configuration** that achieved the best validation/test accuracy is:

Hyperparameter / Property	**Exact Value**
Run Identifier	`act_gelu_bs_64_opt_nadam_lr_0.001_wd_0.0005_bn_True`
Architecture	`VGG6 ([64, 64, 'M', 128, 128, 'M'] + AvgPool(1,1) + Linear(128,`
Activation Function (`--activation`)	`gelu`
Batch Normalization (`--batch-norm`)	`true`
Optimizer (`--optimizer`)	`nadam`
Mini-Batch Size (`--batch-size`)	64
Number of Epochs (`--epochs`)	8
Initial Learning Rate (`--lr`)	`0.001` (with CosineAnnealingLR schedule)
Momentum (`--momentum`)	`0.9`
Weight Decay (`--weight-decay`)	`0.0005`
Data Augmentation (`--use-cutout`)	`RandomCrop(32, pad=4) + RandomHorizontalFlip() + Cutout(1, 16)`
Random Seed (`--seed`)	42
Saved Checkpoint Path	`checkpoints/best_vgg6_cifar10.pth`
Reproduced Best Test Top-1 Accuracy	**67.02%** (**Test Loss:** `0.9378`)

Table 4: Single best VGG6 configuration verified on the CIFAR-10 test set.

5 Question 5: Reproducibility and Repository (10 Points)

1. **Modular Codebase Separation (`src/`):** Dataset transforms (`src/dataset.py`), VGG6 architecture (`src/model.py`), training loop (`src/train.py`), standalone test evaluation (`src/evaluate.py`), and seeding utilities (`src/utils.py`) are separated into clean modules rather than a single monolithic script or notebook.

2. **Exact Commands to Verify and Reproduce:**

```
# 1. Verify the saved trained model checkpoint on the 10,000-image CIFAR-10 test set:
python -m src.evaluate --checkpoint checkpoints/best_vgg6_cifar10.pth --seed 42

# 2. Re-train the exact best configuration from scratch with deterministic seed=42:
python -m src.train --activation gelu --optimizer nadam --batch-size 64 --epochs 8
  --lr 0.001 --momentum 0.9 --weight-decay 0.0005 --batch-norm true --use-cutout true
  --subset-size 15000 --seed 42 --save-checkpoint checkpoints/best_vgg6_cifar10.pth
```

3. **GitHub Repository (Including Trained Checkpoint `checkpoints/best_vgg6_cifar10.pth` & `README.md`):**
https://github.com/cs25m513/CS6886W-Assignment1-VGG6